

Expressions

Amir Jamalian



Gisma
University
of Applied
Sciences





Expressions

- Programs are made up of expressions
 - Which describe to the computer how to combine pieces of data
 - Expressions, such as $3 * 4$, are evaluated by the computer
- The grammar rules of a programming language are rigid
 - The computer will show a `SyntaxError` error if an expression differs from its prescribed expression structures
 - The *Syntax* of a language is its set of grammar rules
- Small changes to an expression can change its meaning entirely
 - $3 * 4$ is not equal to $3 ** 4$
 - It is a semantic error

```
In [6]: 3 *
```

```
File "/tmp/ipykernel_4622/2069723010.py", line 1
```

```
3 *
```

```
^
```

```
SyntaxError: invalid syntax
```



Operators

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators



Arithmetic Operators

- Python expressions obey the same familiar rules of precedence as in algebra
 - Parentheses can be used to group together smaller expressions within a larger expression

Expression Type	Operator	Example	Value
Addition	+	2 + 3	5
Subtraction	-	2 - 3	-1
Multiplication	*	2 * 3	6
Division	/	7 / 3	2.66667
Integer Division	//	7 // 3	2
Remainder	%	7 % 3	1
Exponentiation	**	2 ** 0.5	1.41421



Assignment Operators

[Hunt 2019]

https://www.w3schools.com/python/python_operators.asp

5

- Compound operators combine together

- A numeric operation (such as add)
- With the assignment operator

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3



Names (i.e., Variables)

- Names are given to values in Python using an assignment statement
 - The value of the expression to the right of = is assigned to the name
 - `a = 3 * 4`
 - A previously assigned name can be used in another expression
 - `b = 2 * a`
- Names must start with a letter, but can contain both letters and numbers
 - A name cannot contain a space
 - It is common to use an underscore character `_` to replace each space
 - It is up to the programmer to choose names that are easy to interpret
 - Typically, more meaningful names can be invented than `a` and `b`

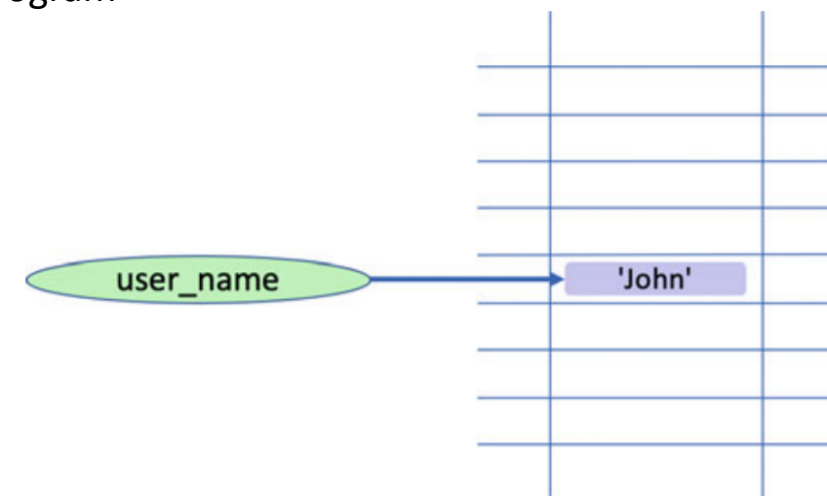


Names (i.e., Variables)

[Hunt 2019]

7

- The names are actually references to a particular space in memory
 - `user_name = "John"`
 - The name `user_name` is acting as a label for an area of memory which holds the value "John"
- A name is also called a variable
 - Because the value it references in memory can vary during the lifetime of the program
 - If we want to, we can reuse the variable to hold another value
 - `user_name = "Alice"`
 - A variable is a named area of the computers' memory
 - It can hold data





Call Expressions

- Call expressions invoke functions, which are named operations
 - The name of the function appears first, followed by expressions in parentheses
 - `abs(-12)`
 - `max(2, 2 + 3, 4)`
 - The max function is called on three arguments
 - The value of each expression within parentheses is passed to the function
 - The function returns the final value of the full call expression
 - The max function can take any number of arguments and returns the maximum
- Operators and call expressions can be used together in an expression
 - `x = 2 + abs(-12)`



Built-In Functions

- A few functions are available by default
 - The built-in functions are listed here <https://docs.python.org/3/library/functions.html>
- The list of built-in functions of Python includes many functions
 - That are never needed in data science applications
 - We will learn and work with the most important functions in context



Expressions vs Statements

- Expression is essentially a computation that generates a value
 - It can be a combination of values and functions
 - `3 * 4 + abs(-12)`
- Statement is just a standalone unit of execution and does not return anything
 - It can be a single or multiple lines of code
 - `print("Hello World!")`





- Comments are sections of a program that are ignored by the Python interpreter
 - They are not executable code
 - It is common practice to add comments to code
 - It helps anyone reading the code to understand what the code does
- A comment is indicated by the # character in Python
 - Anything following that character to the end of the line will be ignored by the interpreter

```
# Below, we multiply 3 by 4  
x = 3 * 4    # This is also another comment
```